

Estos resultados ya nos muestran algo muy importante: **el Z-score es técnicamente viable, pero no todas las variables se comportan bien cuando lo calculamos.**

Primero, una observación pequeña: en tu tabla de cobertura de TRAIN aparecen 270033 CASES y 2700330 CONTROLS. Por consistencia con la cohorte congelada y con los porcentajes, entiendo que son 27,033 y 270,330; parece solo un cero adicional al copiar la tabla.

La cobertura del Z-score es razonable. En TRAIN, aproximadamente 81% de los CASES tienen `active_accounts_z`, 92% tienen `balance_z` y 83% tienen `utilization_z`; en CONTROLS la disponibilidad es menor, pero todavía considerable. Los flags además están funcionando correctamente porque todos los `flag_but_null = 0`.

El problema aparece en la **distribución del Z**.

Para `active_accounts_z`, la cola es relativamente manejable:

P95 4.57
P99 6.98
P99.9 15.02
MAX 92.52

Hay extremos, pero no es sorprendente que aparezcan cuando una persona históricamente tenía poca variación en número de cuentas y luego cambia bruscamente. Este Z-score todavía me parece un challenger razonable.

Para `avg_utilization_z`:

P95 4.84
P99 10.12
P99.9 33.76

MIN -708.99
MAX 2,540.34

Aquí ya vemos claramente el efecto de un `SD prior` muy pequeño. Aunque capeamos `utilization` antes de calcular el SD, si el comportamiento histórico es casi constante:

$SD_{prior} \approx 0$

entonces:

$\frac{Recent - Prior}{SD_{prior} / \sqrt{3}}$

puede explotar.

Eso no invalida `utilization_z`, pero significa que **no usaría el Z raw directamente en una regresión logística.**

El caso más problemático es `balance_z`:

P95 5.82
P99 32.86
P99.9 333.94

MIN -4.31×10^{15}
MAX 1.56×10^{15}

Esos extremos no representan un cambio económicamente interpretable de “trillones de desviaciones estándar”. Son básicamente una consecuencia numérica de tener un `balance_prior_sd` extremadamente cercano a cero.

Por eso **yo sacaría `balance_z` del challenger en su forma actual**. Mantendría perfectamente:

`avg_balance_recent_3m`
`avg_balance_prior_9m`
`balance_change_vs_baseline`

pero no `balance_z_3m`.

Hay además una anomalía que quiero verificar en `active_accounts_z`: reportaste:

P0.1 = -1.327906
P1 = -5.842374
P5 = -3.476787

Eso no puede ser el orden correcto de percentiles, porque debería cumplirse:

$P_{0.1} \leq P_1 \leq P_5$

Podría ser simplemente un valor copiado en la columna equivocada. No afecta la conclusión todavía, pero conviene comprobarlo.

Antes de decidir cómo capear el Z

No quiero utilizar arbitrariamente algo como:

Z between -10 and +10

ni introducir un `SD floor = 0.01`.

Podemos hacer algo mucho más defendible: **entender qué tan pequeños son realmente los SD históricos** y aprender cualquier regla únicamente en TRAIN.

Ejecutaría este QA:

SELECT

```
/* =====  
ACTIVE ACCOUNTS SD > 0  
===== */
```

```
percentile_approx(  
  CASE  
    WHEN active_accounts_prior_sd > 0  
    THEN active_accounts_prior_sd  
  END,  
  ARRAY(0.001,0.005,0.01,0.05,0.10,0.50)  
) AS active_sd_percentiles,
```

```
/* =====  
BALANCE SD > 0  
===== */
```

```
percentile_approx(  
  CASE  
    WHEN balance_prior_sd > 0  
    THEN balance_prior_sd  
  END,  
  ARRAY(0.001,0.005,0.01,0.05,0.10,0.50)  
) AS balance_sd_percentiles,
```

```
/* =====  
AVG UTILIZATION SD > 0  
===== */
```

```
percentile_approx(  
  CASE  
    WHEN utilization_prior_sd > 0  
    THEN utilization_prior_sd  
  END,  
  ARRAY(0.001,0.005,0.01,0.05,0.10,0.50)  
) AS avg_util_sd_percentiles,
```

```
/* =====  
WEIGHTED UTILIZATION SD > 0  
===== */
```

```
percentile_approx(  
  CASE  
    WHEN weighted_util_prior_sd > 0  
    THEN weighted_util_prior_sd
```

```
        END,  
        ARRAY(0.001,0.005,0.01,0.05,0.10,0.50)  
    ) AS weighted_util_sd_percentiles
```

```
FROM  
    bu_frd.model_behavior_features_z_2022_2025
```

```
WHERE model_split = 'TRAIN';
```

Y además quiero cuantificar específicamente SD casi cero:

```
SELECT
```

```
    COUNT(*) AS train_parties,
```

```
    /* Balance */
```

```
    SUM(  
        CASE WHEN balance_prior_sd > 0  
              AND balance_prior_sd < 0.01  
              THEN 1 ELSE 0 END  
    ) AS balance_sd_lt_001,
```

```
    SUM(  
        CASE WHEN balance_prior_sd >= 0.01  
              AND balance_prior_sd < 1  
              THEN 1 ELSE 0 END  
    ) AS balance_sd_001_to_1,
```

```
    /* Utilization */
```

```
    SUM(  
        CASE WHEN utilization_prior_sd > 0  
              AND utilization_prior_sd < 0.001  
              THEN 1 ELSE 0 END  
    ) AS avg_util_sd_lt_0001,
```

```
    SUM(  
        CASE WHEN utilization_prior_sd >= 0.001  
              AND utilization_prior_sd < 0.01  
              THEN 1 ELSE 0 END  
    ) AS avg_util_sd_0001_to_001,
```

```
    /* Weighted utilization */
```

```
    SUM(  
        CASE WHEN weighted_util_sd > 0  
              AND weighted_util_sd < 0.001  
              THEN 1 ELSE 0 END  
    ) AS weighted_util_sd_lt_0001,
```

```

CASE WHEN weighted_util_prior_sd > 0
      AND weighted_util_prior_sd < 0.001
      THEN 1 ELSE 0 END
) AS weighted_util_sd_lt_0001,

SUM(
CASE WHEN weighted_util_prior_sd >= 0.001
      AND weighted_util_prior_sd < 0.01
      THEN 1 ELSE 0 END
) AS weighted_util_sd_0001_to_001

FROM
bu_frd.model_behavior_features_z_2022_2025

WHERE model_split = 'TRAIN';

```

También me falta en tus resultados la distribución de **weighted_utilization_z_3m**. La sacaría con:

```

SELECT

percentile_approx(
  weighted_utilization_z_3m,
  ARRAY(0.001,0.01,0.05,0.50,0.95,0.99,0.999)
) AS weighted_util_z_percentiles,

MIN(weighted_utilization_z_3m)
  AS weighted_util_z_min,

MAX(weighted_utilization_z_3m)
  AS weighted_util_z_max

FROM
bu_frd.model_behavior_features_z_2022_2025

WHERE model_split = 'TRAIN';

```

Con la evidencia actual, mi selección provisional queda así

Variable	Fórmula actual		Z challenger
Deterioration	✓	✗	
Adverse	✓	✗	
Active accounts	✓	✓	
Balance	✓	✗	raw Z

Past due	✓	✗
Delinquency share	✓	✗
Avg utilization	✓	✓, con tratamiento de cola
Weighted utilization	✓	✓, pendiente confirmar distribución

Esto en realidad es un buen resultado. El objetivo del experimento no era demostrar que Z-score funciona para todo, sino identificar **dónde agrega una representación estadísticamente útil**.

Y hasta ahora los datos están diciendo que **Recent - Prior** es mucho más robusto como representación general, mientras que Z-score puede ser un complemento específico para **active_accounts** y utilization.

Cuando tengamos esos tres QA de SD y weighted Z, podemos cerrar 23E y pasar al **Paso 24: medir señal predictiva univariada en TRAIN**, que es donde por fin veremos si el Z-score realmente mejora algo respecto a nuestra fórmula actual.